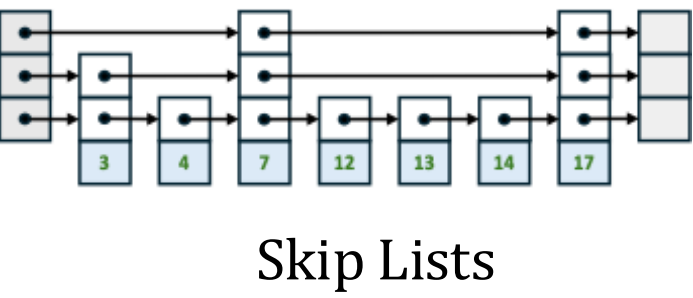
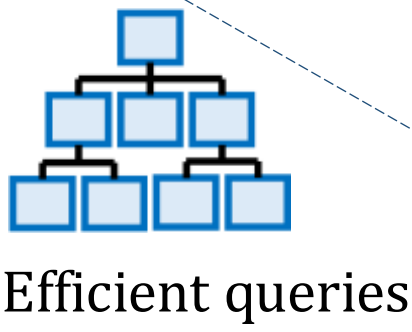
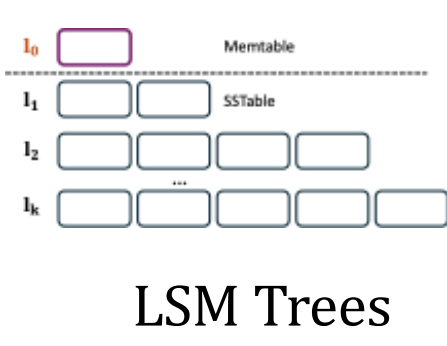
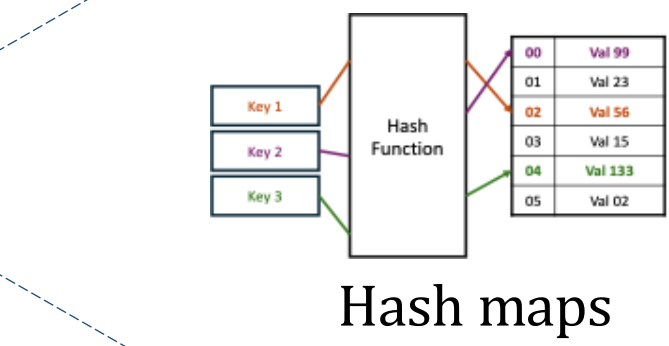
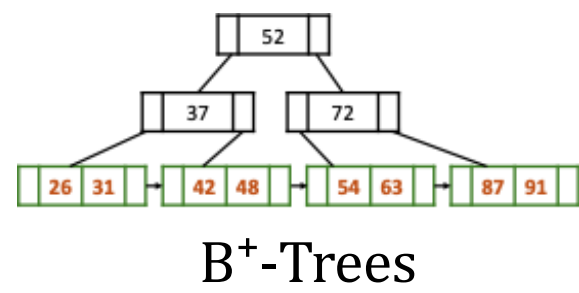
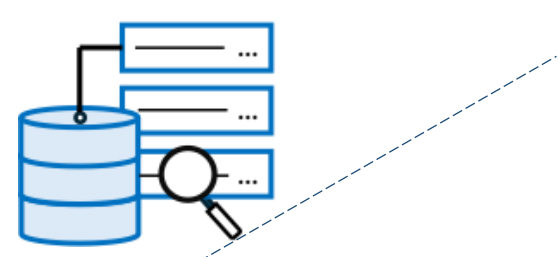
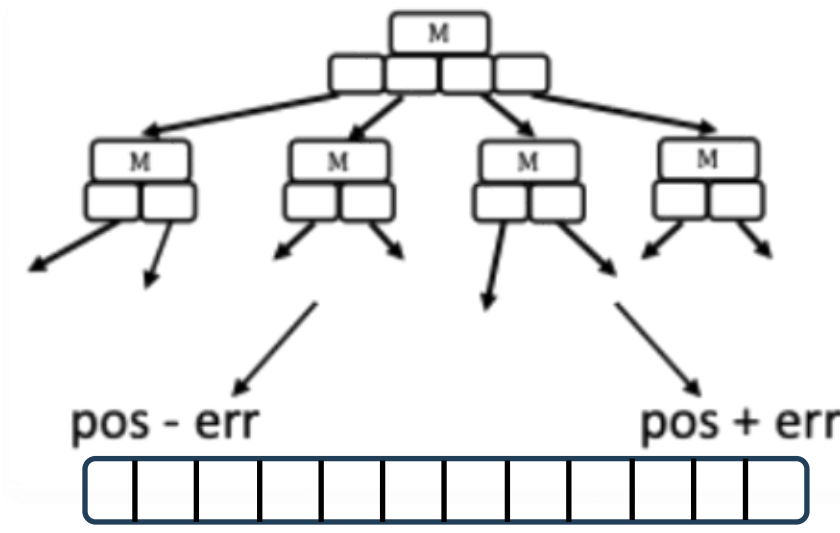


Indexes in Data Systems

Organize data



Learned Indexes



Lower Memory

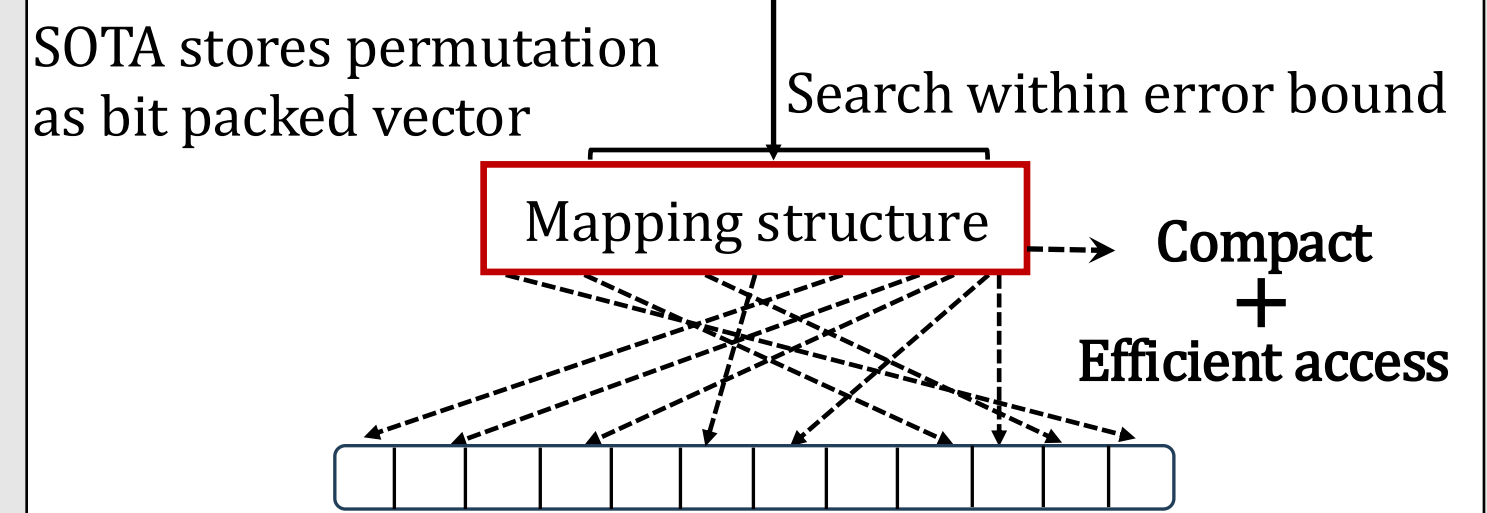
Faster Access

Data assumed to be sorted! → Instead store a permutation!

Permutation	9	6	2	8	3	7	1	5	4	0
Base data	923	430	98	265	810	578	68	345	114	56

Broader Vision

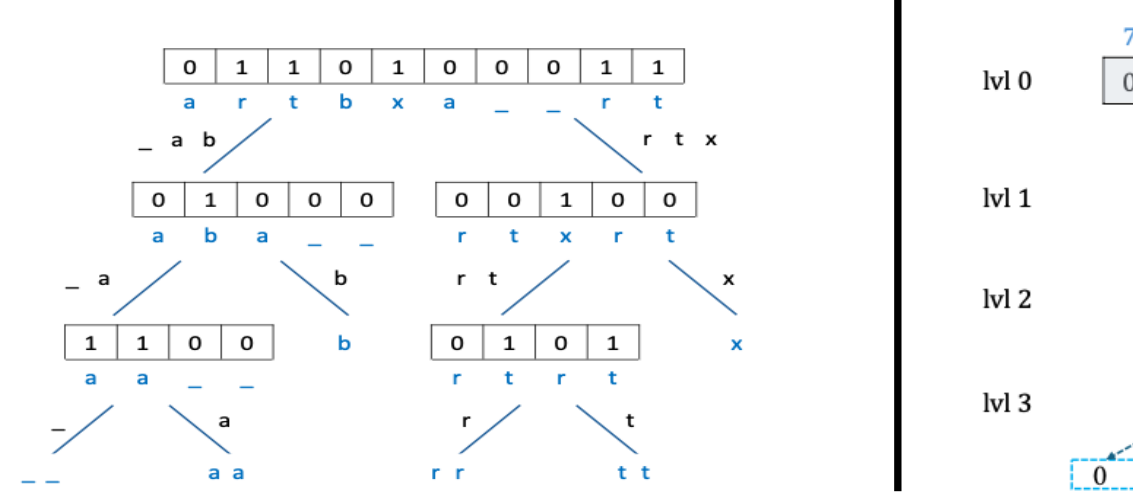
[LSI, aiDM '22]



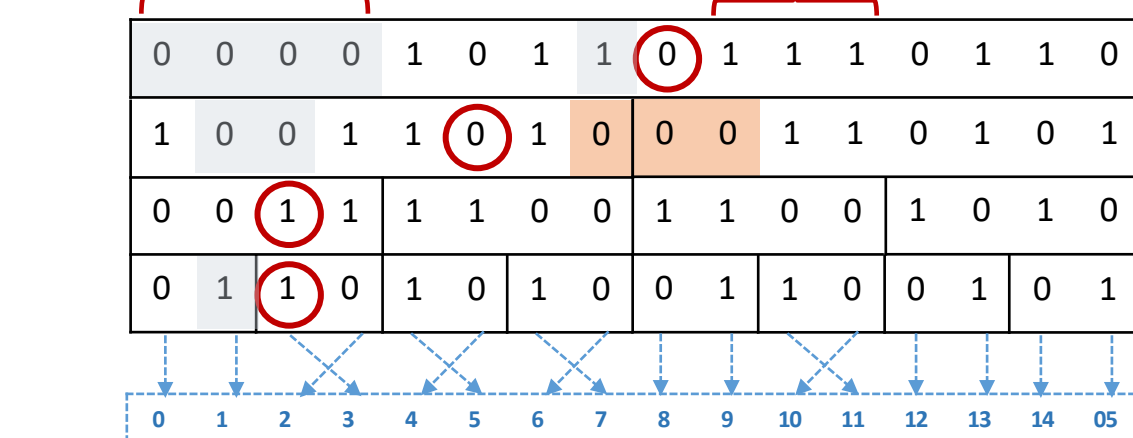
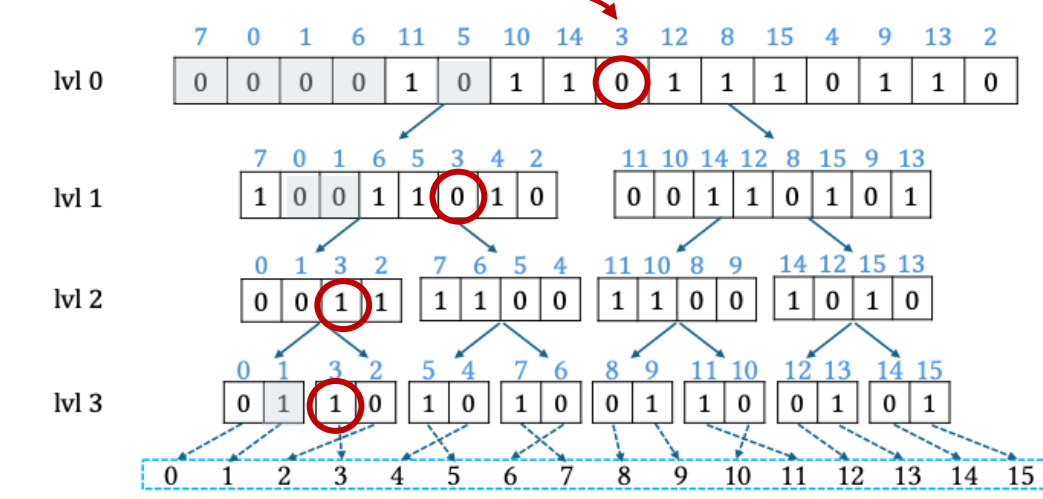
Access(Mapping structure) << Access(Traditional index)

Extending Wavelet Trees to Integer Wavelet Tree (IWT)

Wavelet Tree



Query index 8



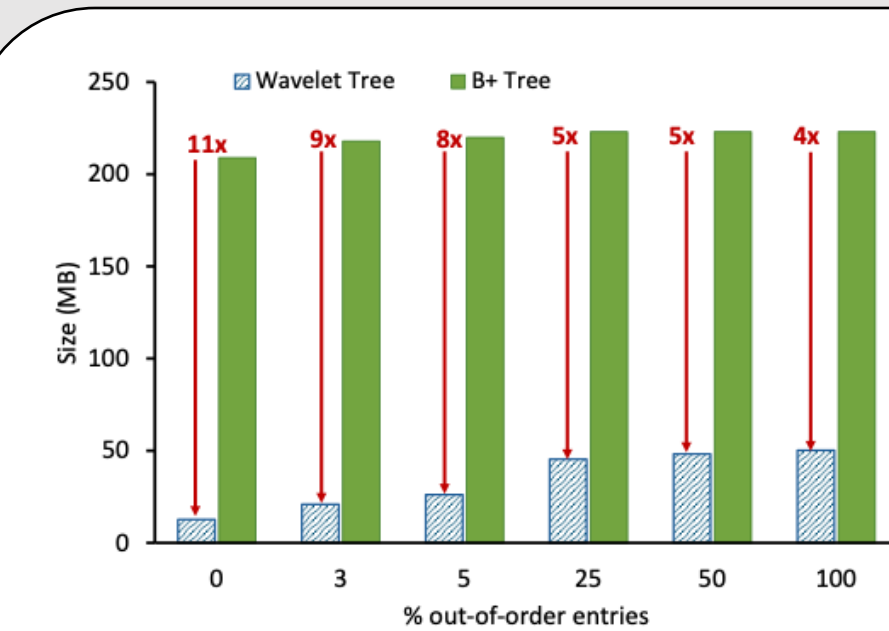
Integer Wavelet Tree is stored as a flattened matrix

Access in Wavelet Tree:

Value determines which segment to scan

Rank denotes which element within segment

Results of IWT and Extending to T-Way Tree



IWT upto 11x smaller
Order of magnitude slower

To make access faster, store rank as well as indexed entries

L0	1	3	1	0	0	2	0	2	1	2	3	2	1	3	3	0
L1	0	3	1	2	1	0	3	2	3	1	2	0	0	2	1	3

Indexed Entries

R0	0	0	1	0	1	0	2	1	2	2	1	3	3	2	3	3
R1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Rank

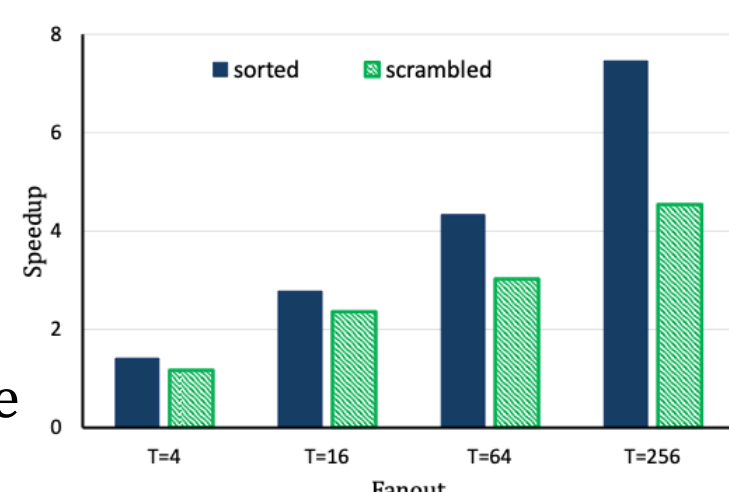
Representative 4-Way Wavelet Tree

Access Efficiency with T-Way Tree

Design Considerations

Can extending fanout proportionally speedup access in IWT?

How can we balance space with extra rank stored?

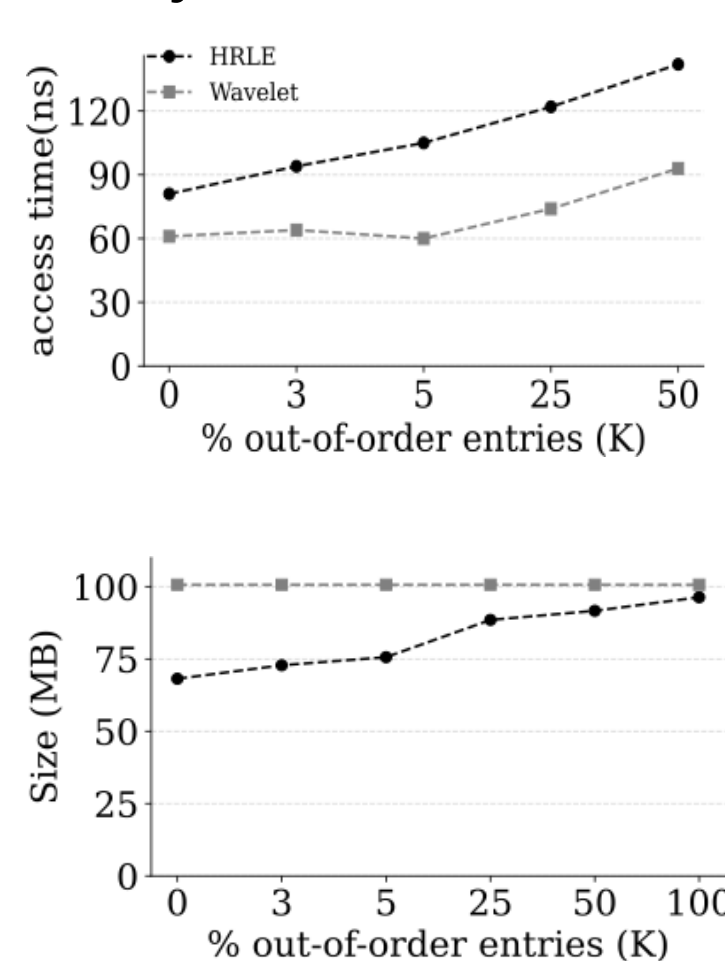


Speedup of T-Way Tree against a B+-Tree

Increasing fanout decreases access cost proportionally

Balancing Space and Access Efficiency

Hybrid RLE



For each level, divide the elements into 128 element blocks

At each block store raw/ RLE based on data requirements

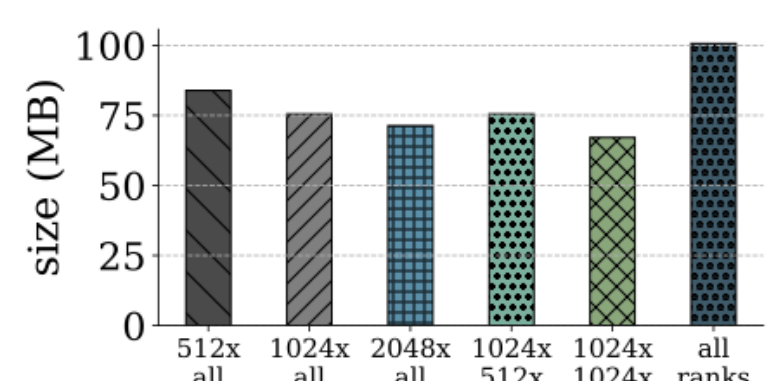
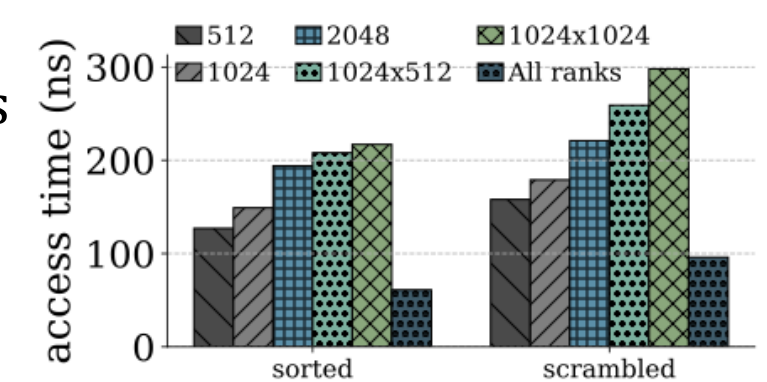
Selective Rank Storage

L0 → wider rank values (n/256 per node)

Store rank every B positions

↓ B → fast access

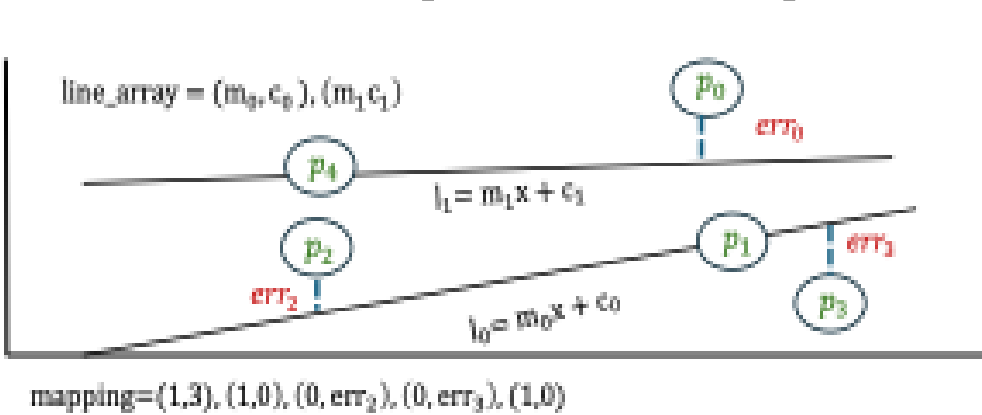
↑ B → less space



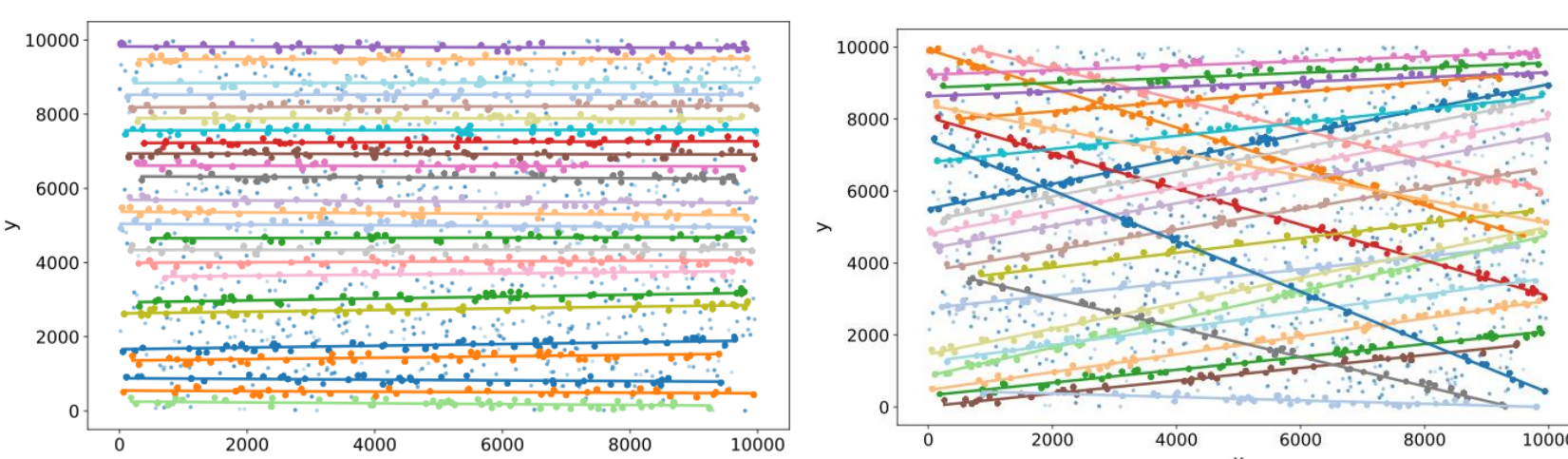
Selective rank storage trades space for access speed

Extending to a new Data Structure – Constellation Maps

Permutation as points in 2-D space



Representation of 10000 points mapped to lines (25)



Recursive LR

Extremely fast
Linear pattern data (errors are in Gaussian dist. around line)

RANSAC

Slow - speed depends on hyperparameters
Handles arbitrary patterns with outliers

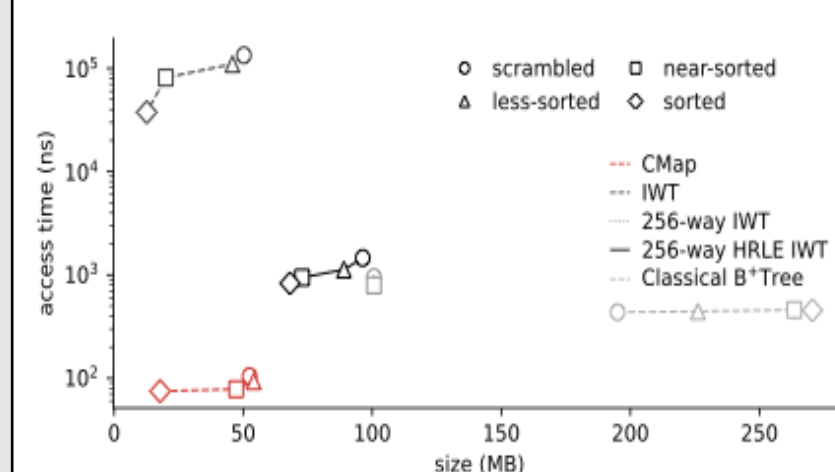
Putting it together

Next Steps

Hybrid method of storage (hashmap + constellation map)

Parallelize algorithms like RANSAC to ensure fast updates and inserts

Handle updates via buffered LSM-like delta and merge strategy



CMap results demonstrate effective space-time tradeoff